ECE 408

第 13 讲

Atomic Operations and Histogramming

目录

1 并发更新与原子操作 **1**
1.1 读—改—写与更新丢失 . . . 1
1.2 互斥、线程束与执行进度 . . . 2
1.3 原子更新的保证与边界 . . . 2
1.4 CUDA 原子接口与比较交换 . . . 3
1.5 用比较交换构造原子加法 . . . 4

2 直方图与并行数据划分 **5**
2.1 桶映射与计数不变量 . . . 5
2.2 输入划分与输出地址的关系 . . . 6
2.3 网格跨步核函数与完整启动条件 . . . 7

3 原子操作的延迟与吞吐 **8**
3.1 同一地址上的串行服务 . . . 8
3.2 从显存到二级缓存和共享内存 . . . 9
3.3 性能优化的两个方向 . . . 10

4 线程块私有化与共享内存 **10**
4.1 局部统计与全局结果的等价性 . . . 10
4.2 初始化、局部统计与全局合并 . . . 11
4.3 合并阶段的并发与同步边界 . . . 13
4.4 原子操作数量与资源开销 . . . 13
4.5 桶数超过共享内存容量时 . . . 14

5 线程粗化与性能权衡 **15**
5.1 增加每个线程处理的输入数 . . . 15
5.2 粗化的数量例子与并行度代价 . . . 15
5.3 线程级局部聚合 . . . 16
5.4 访存、分支和复用的整体判断 . . . 18

6 例题与完整推导 **19**
6.1 从输入划分到最终直方图 . . . 19
6.2 由浮点吞吐率反推 L2 原子操作比例 . . . 20
6.3 实现检查与关键关系 . . . 21

1 并发更新与原子操作

1.1 读—改—写与更新丢失

共享计数器的更新通常包含读取旧值、计算新值和写回三个步骤，称为读—改—写（read-modify-write）。例如，`++bins[b]` 在逻辑上执行“读出第 b 个计数器，将它加一，再写回”。即使源程序只写了一条语句，也不能据此认为其他线程无法插入执行。

设共享计数器初值为 0，线程 A 与线程 B 各执行一次加一。每个线程分别持有局部变量 r_A、r_B；这些变量只记录该线程读取到的旧值。若 A 完成整个更新后 B 才读取，则 A 读到 0、写入 1，B 随后读到 1、写入 2。交换两个线程的先后顺序，最终也得到 2。然而，表1中的交错执行会丢失一次更新。

表 1 两个线程各自完成加一，最终计数却只有一

时刻	线程 A	线程 B	共享值
1	读取旧值，$r_A = 0$		0
2	计算 $r_A + 1 = 1$		0
3		读取旧值，$r_B = 0$	0
4	写回 1		1
5		计算 $r_B + 1 = 1$	1
6		写回 1	1

两个线程都完成了自己的指令，但它们根据同一个旧值计算，第二次写回覆盖了第一次更新。问题出在共享状态的访问没有形成正确的先后关系。多个线程并发访问同一内存位置、至少一次为写入，并且这些冲突访问缺少适当的同步或原子保证时，就存在数据竞争（data race）。上表只展示一种直观的错误交错；含数据竞争的程序不能依靠这几种手工列举的结果来保证行为。

类似的问题也出现在状态占用中。若用 0 表示座位空闲、1 表示已占用，两个请求都可能先读到 0，再分别写入 1，于是它们都认为自己取得了同一个座位。计数需要保护“读取后累加”的整体，占用操作需要保护“检查条件后修改”的整体。两者都要求把依赖旧状态的判断或计算与相应写回联系起来。

是否发生冲突由最终访问的地址决定。两个线程读取不同输入，并不保证它们写入不同输出。例如，不同键可能散列到同一个表项，不同顶点也可能向共同邻居累加贡献。直方图正具有这种结构：输入位置可以独立划分，而输入的值可能把多个线程映射到同一个计数器。

1.2 互斥、线程束与执行进度

互斥（mutual exclusion）要求同一时刻至多有一个参与者执行受保护的更新。将读取、计算和写回放入临界区（critical section），并在进入前获取锁、退出后释放锁，可以使整个更新按某种顺序完成。例如，对同一把互斥锁，只有 A 释放后 B 才能进入，或者只有 B 释放后 A 才能进入。两种顺序都避免了上面的覆盖问题。

这种方法依赖一个进度条件：获得锁的线程必须能够继续执行并最终释放锁。在传统的线程束（warp）共同推进模型中，同一束中的一个线程可能获得锁，其余线程则停留在等待锁的循环中。若执行调度一直停留在等待分支，使持锁线程的临界区与释放语句无法得到执行，就出现循环等待：等待者需要锁被释放，持锁者又无法越过当前执行路径去释放锁。这是把简单自旋锁直接搬入这种执行模型时可能发生死锁的原因。

上述死锁分析以线程束内的具体调度模型为前提。独立线程调度（independent thread scheduling）改变了线程束内线程推进的方式，因此分析锁时仍需同时检查参与线程的执行方式、内存可见性以及持锁者的进度。对于单个计数器加一这样的操作，可以直接使用硬件支持的原子更新，避免手工管理锁住和释放的临界区。

1.3 原子更新的保证与边界

原子操作（atomic operation）将指定地址上的读—改—写作为一个不可分割的操作完成。对于同一地址上、作用域相互覆盖的冲突原子操作，每次更新都必须看到某个完整的先前状态；硬件不会让另一原子更新插入这次操作的中间阶段。因此，同地址的冲突更新在效果上形成一个串行顺序。

若计数器初值为 h_0，所有 m 次更新都通过原子加一完成，而且没有溢出或其他未同步访问，那么最终值就是 $h_0 + m$。执行顺序可以未知，结果仍然确定，因为整数加法满足交换律和结合律。对于其他更新，顺序仍可能影响结果。例如，对同一位置分别执行原子交换，写入 3 和写入 7，最终值取决于哪个交换最后完成。

原子保证只覆盖该操作指定的状态转移。对两个不同计数器先后执行两次原子加法，并不会自动把二者合成一个不可分割的事务；另一个线程可能观察到其中一次已经完成、另一次尚未完成。对单个计数器的原子加法也不提供“所有线程都已完成”的通知。后面在共享直方图中使用的块内同步，负责保证一个阶段全部结束后才开始下一阶段；原子加法则负责该阶段内每次冲突更新的正确性。

原子性与顺序

同一地址的原子更新不会互相覆盖掉未完成的中间结果，但线程之间谁先更新通常没有预定顺序。计数结果的确定性还依赖更新运算本身；完整结果应在所有更新结束后读取。

1.4 CUDA 原子接口与比较交换

CUDA 提供由编译器识别的内建原子函数。`atomicAdd` 接受目标地址和增量两个参数，原子地读出旧值、加上增量、写回结果，并返回更新前的旧值。例如，一次调用向计数器加入 3，若原子动作读到 4，它就写入 7，并向调用者返回 4。多个线程争用同一地址时，返回值也服从实际形成的串行顺序。

`atomicAdd` 有不同数据类型的重载，常见类型包括整数与浮点数；具体重载的支持取决于设备和数据类型。计数问题通常使用整数。其他原子原语包括减法、最小值、最大值、按位与、按位或、按位异或、交换以及比较交换。对于可交换的整数累加，原子操作保持计数正确；浮点加法还受到舍入和累加顺序的影响。

比较交换（compare-and-swap，CAS）把“检验当前位置是否仍为预期值”与“条件成立时写入新值”合成一个原子动作。用 v 表示操作开始时该地址的值，e 表示预期值，d 表示希望写入的值，其状态变化为

$$v_{\text{after}} = \begin{cases} d, & v = e, \\ v, & v \neq e. \end{cases} \tag{1}$$

在抽象描述中，可以让 CAS 返回“成功”或“失败”。CUDA 的 `atomicCAS` 接口依次接受目标地址、预期值和目标新值三个参数。无论替换是否成功，它都返回操作之前观察到的旧值。因此，返回旧值等于预期值，才是替换成功的判断条件。

这一区别会直接影响程序逻辑。如果旧值为 0、预期值也为 0，替换成功时返回 0；直接将返回值当作布尔量，会把这次成功当成失败。反过来，如果当前值为 5、预期值为 0，替换失败时返回 5；将其转换为布尔量又会得到真。普通的条件判断加赋值本身也无法实现式(1)要求的不可分割性，真正的保证来自原子原语。

对座位占用状态，可以执行 `old = atomicCAS(&seat, 0u, 1u)`。若返回 0，说明本次调用取得了此前空闲的座位；若返回 1，说明调用时座位已被占用。两个并发请求都使用这一原语时，只有一个能够把同一次空闲状态成功改为占用状态。

无后缀的传统 CUDA 原子接口采用设备作用域；对于全局计数器，这允许同一设备中不同线程块共同原子更新。它们不会自动建立其他地址的读写顺序。共享内存对象则属于各自线程块，只有该块中的线程参与访问。地址必须有效并满足类型对齐要求，初始化需要先于

并发更新，所有冲突访问也必须遵守相容的原子与同步规则。

1.5 用比较交换构造原子加法

CAS 可以构造“根据旧值计算新值”的更新。线程先取得一个估计值，计算目标值，然后尝试仅在共享值仍与估计相等时写入。如果比较时共享值与估计值不同，CAS 会失败并返回新的观测值；当前线程据此重新计算和重试。这样，线程私有的计算可以被打断，最终提交更新的动作仍然是原子的。

代码1用无符号整数实现这一过程。函数名与 CUDA 内建的 `atomicAdd` 区分开，且整个实现中的共享访问都使用 CAS。初始调用 `atomicCAS(address, 0u, 0u)` 只用于取得当前值：当前值为 0 时写回相同的 0，否则保持原值。

代码 1 由 CAS 构造无符号整数原子加法

```
__device__ unsigned int atomic_add_cas(
    unsigned int* address, unsigned int value)
{
    unsigned int old = atomicCAS(address, 0u, 0u);
    while (true) {
        unsigned int assumed = old;
        old = atomicCAS(address, assumed, assumed + value);
        if (old == assumed) {
            return assumed;
        }
    }
}
```

变量 `assumed` 保存本轮准备提交时的预期旧值；`old` 在 CAS 返回后变为该原子动作实际观察到的值。若二者相同，说明比较时共享值确实等于 `assumed`，写入的正是这个当前值加上 `value`，函数返回此次更新前的值。若二者不同，本次 CAS 没有修改共享状态，下一轮直接使用返回值作为新估计。

例如，初始计数为 0，A、B 各加 1，两者都先估计为 0。A 的 CAS 将 0 改成 1，返回 0，因此成功。B 随后尝试把 0 改成 1，但当前值已经是 1，于是 CAS 返回 1 且不写入。B 下一轮改为尝试把 1 改成 2，成功后返回 1。最终计数为 2。

正确性的关键是成功 CAS 的时刻。在线程准备估计值或计算加法期间，其他线程都可以继续更新；只有当提交时的共享值满足比较条件，当前计算才会生效。因此，函数整体可以被看作在成功 CAS 时刻完成了一次加法。竞争越激烈，失败重试可能越多，不能为每个线程保证固定的重试次数或取得更新机会的顺序。已有合适的内建原子加法时，应直接使用内建函数；这个构造用于解释 CAS 如何提供更一般的更新机制。

无符号整数在溢出时按其位宽回绕。若把它作为通常意义上的计数器，仍须保证实际计数不超过可表示范围。原子性保证更新不会相互丢失，计数容量则决定最终数值能否完整表示真实数量。

2 直方图与并行数据划分

2.1 桶映射与计数不变量

直方图（histogram）统计数据落入不同类别或区间的次数。设输入为 $x_0, x_1, \dots, x_{N-1}$，共有 B 个桶（bin），桶映射 $b(x)$ 把每个有效输入映射到 $\{0, 1, \dots, B-1\}$。第 j 个桶的计数为

$$H_j = \sum_{i=0}^{N-1} \mathbf{1}\{b(x_i) = j\}, \qquad j = 0, \dots, B-1, \tag{2}$$

其中指示函数 $\mathbf{1}\{\mathcal{E}\}$ 在条件 $\mathcal{E}$ 成立时取 1，否则取 0。这里每个输入恰好进入一个桶，因此 $\sum_{j=0}^{B-1} H_j = N$。这一总计数不变量适合检查输入遗漏、重复处理或更新丢失，不过总数相等还不足以保证各桶都正确。

对于一个字节表示的灰度值，$x_i \in \{0, \dots, 255\}$，可以取 $B = 256$、$b(x_i) = x_i$。若图像宽度为 W_{img}、高度为 H_{img}，输入长度为 $N = W_{\text{img}} H_{\text{img}}$；随后只需按一维数组遍历。以下代码使用 `unsigned char` 表示输入字节，避免普通 `char` 的符号性使较大字节值变成负下标。

桶也可以表示数值区间。例如，将 256 种灰度值分成四个宽度为 64 的区间，可以取 $b(x) = \lfloor x/64 \rfloor$。此时 0 与 63 进入桶 0，64 进入桶 1，255 进入桶 3。映射规则改变了输出桶数，但每个有效像素仍恰好贡献一次计数。

字母频率也是直方图。对输入字符串

```
Programming Massively Parallel Processors
```

若统一大小写、跳过空格，并令字母 A 到 Z 对应 26 个桶，则共有 38 个有效字母。表2列出所有非零计数；其余字母计数为 0。这里“跳过空格”是桶映射前的数据筛选规则，所以总计数应等于有效字母数。

直方图可以概括图像灰度、类别频数或数值区间分布。不同应用的差别主要在桶映射，而并行计算面对的共同问题是：每个输入贡献一个增量，多个增量可能同时指向同一个桶。

表 2 字母直方图的完整非零计数

字母	A	C	E	G	I	L	M
次数	4	1	3	2	2	4	3
字母	N	O	P	R	S	V	Y
次数	1	3	3	5	5	1	1

2.2 输入划分与输出地址的关系

将输入分给不同线程，可以保证每个输入位置只被处理一次，但输出桶仍由输入值决定。例如，四个线程当前读到的字母分别为 P、M、M、E，则这轮需要给 P 和 E 各加 1，给 M 加 2。处理两个 M 的线程即使读取了不同位置，也会对同一个输出地址形成竞争。

最直接的并行方案让每个线程读入一个或多个数据，并对相应桶执行原子加一。原子操作解决输出的冲突，输入读取方式则影响全局内存访问效率。这两个方面可以分别设计：相邻线程可以读取相邻数据，但它们所更新的桶仍可能离散或高度集中。

设网格含 G 个线程块，每块有 T 个线程，总线程数为 $P = GT$。为便于比较，暂设每个线程恰好处理 c 个输入，即 $N = Pc$。一种划分方式给全局编号为 g 的线程分配一个连续的长度为 c 的区间，第 k 轮读取位置 $gc + k$。虽然单个线程连续读取，但是同一轮中相邻线程的地址间隔为 c，会使一个线程束的访问分散。

另一种方式让线程 g 在第 k 轮读取 $g + kP$。于是同一轮中线程 g、$g + 1$ 读取相邻位置，而每个线程在不同轮之间跨过 P 个元素。表3用 $P = 4$、$c = 4$ 给出完整地址序列。

表 3 四个线程处理十六个输入的两种分配

线程	每线程连续区间	每轮相邻线程访问相邻位置
0	0, 1, 2, 3	0, 4, 8, 12
1	4, 5, 6, 7	1, 5, 9, 13
2	8, 9, 10, 11	2, 6, 10, 14
3	12, 13, 14, 15	3, 7, 11, 15

在第一轮中，两种方案的线程组分别访问 $0, 4, 8, 12$ 和 $0, 1, 2, 3$。合并访存（coalesced memory access）关注的是同一条访存指令下线程束内各线程地址的组合。后者让访问落在连续地址范围，更容易用少量内存事务完成；实际事务数量还取决于访问宽度、对齐和硬件粒度。四线程表只是缩小的索引示例，相同原则适用于完整线程束。

2.3 网格跨步核函数与完整启动条件

将上面的第二种划分写成循环，得到网格跨步循环（grid-stride loop）。设线程块编号为 r、块内线程编号为 t，则线程的全局编号为 $g = rT + t$，跨步长度为 $P = GT$，依次处理 $g, g+P, g+2P, \ldots$ 中小于 N 的索引。代码2直接对公共直方图做全局原子加法。

代码 2 统计 256 种无符号字节值的基本核函数

```
__global__ void histogram_global(
    const unsigned char* input, std::size_t n,
    unsigned int* hist)
{
    std::size_t i = std::size_t(blockIdx.x) * blockDim.x
                  + threadIdx.x;
    std::size_t stride = std::size_t(gridDim.x) * blockDim.x;

    for (; i < n; i += stride) {
        unsigned int bin = input[i];
        atomicAdd(&hist[bin], 1u);
    }
}
```

长度、索引和跨步长度均使用 `std::size_t`。乘法之前先提升类型，避免先以较窄整数计算乘积，再将已经溢出的结果赋给宽类型。代码需要包含 `<cstddef>` 和 CUDA 运行时头文件。这里的核函数采用一维线程块与一维网格，输入与输出数组互不重叠，输出恰有 256 个计数器。

跨步划分的完整性可以直接证明。任取 $i \in \{0, \ldots, N-1\}$，带余除法给出唯一分解 $i = kP + g$，其中 $0 \le g < P$。因此索引 i 恰由线程 g 在第 k 轮处理，既不会遗漏，也不会重复。末尾不足一轮时，条件 $i < N$ 使无对应输入的线程停止循环。网格线程数可以小于输入数，也可以大于输入数。

核函数只累加，因而一次独立统计开始前必须先将公共直方图清零。若希望跨多批输入累积同一结果，则只在第一批之前清零，后续各批需要遵守同样的访问顺序。对于 32 位无符号计数器，每个桶最终计数应不超过 $2^{32} - 1$；这项要求约束实际计数，与输入索引使用宽整数是两个不同问题。

代码3给出一个完整的异步启动函数。输入已经位于设备内存，调用者提供合法的线程块数和每块线程数。清零与核函数提交到同一个流（stream），使核函数在清零完成后才开始使用输出。

代码 3 同一流中先清零再启动统计

```
cudaError_t launch_histogram_global(
    const unsigned char* input, std::size_t n,
    unsigned int* hist, unsigned int blocks,
    unsigned int threads, cudaStream_t stream)
{
    if (blocks == 0 || threads == 0) {
        return cudaErrorInvalidConfiguration;
    }
    cudaError_t status = cudaMemsetAsync(
        hist, 0, 256 * sizeof(unsigned int), stream);
    if (status != cudaSuccess || n == 0) {
        return status;
    }
    histogram_global<<<blocks, threads, 0, stream>>>(
        input, n, hist);
    return cudaGetLastError();
}
```

返回成功表示清零和启动请求已按上述规则提交，异步核函数此时可能仍在执行。主机消费结果之前，需要等待相应流完成并检查完成状态；若之后在同一流中安排设备上的后续计算，则由流内顺序保证先后关系。该函数在 $N = 0$ 时仍把输出清零，然后跳过核函数启动。

基本算法总共读取 N 个输入元素，执行 N 次全局原子加。不同输入值决定这些更新如何分布到桶上：分布均匀时，许多桶可以同时接受更新；若大量输入具有相同值，同一个计数器就成为热点。原子操作保证后一种情况下仍得到正确结果，但其性能需要进一步分析。

3 原子操作的延迟与吞吐

3.1 同一地址上的串行服务

吞吐率（throughput）表示单位时间完成多少次操作，延迟（latency）表示一次操作从发起到完成经历多久。对普通内存访问，多个独立请求可以重叠，因此一个请求延迟较长，并不必然使总吞吐率等于该延迟的倒数。原子热点的特殊之处在于，很多请求依赖同一个地址上前一次更新的结果。

考虑一个简化模型：同一地址每次原子更新独占服务时间为 τ，下一次冲突更新必须在前一次服务完成后才能开始。对持续饱和的请求序列，这个地址的最高更新速率约为

$$R_{\text{addr}} \le \frac{1}{\tau}. \tag{3}$$

这里 τ 是模型中的持续服务间隔，不是某个线程排队后测到的总等待时间。若同时到达 q 个更新，最后一个请求可能需要等待约 $q\tau$ 才完成，但设备在这段时间内仍以每 τ 完成一次更新的速率工作。

假设这个服务过程需要超过 1000 个时钟周期，那么同一热点地址每周期完成的更新少于 1/1000 次。相对于“每周期一次更新”的理想基准，速率降到了不足千分之一。这个比较的对象是单个热点地址的更新能力；若多个地址可以独立服务，它们仍可能并行贡献总吞吐。

令桶 j 一共接收 H_j 次更新。若采用相同服务时间 τ，即使忽略读取输入、其他桶和所有调度开销，完成统计的时间仍受到最长串行链约束：$T_{\text{hot}} \geq \max_j H_j\tau$。所有输入进入同一个桶时，这条链长为 N；输入接近均匀分布时，每个桶大约只承担 N/B 次更新。这个下界说明了输入分布如何影响竞争，但多个桶仍可能共享内部执行资源，因此它本身不是完整的设备性能预测。

更多线程有助于提出更多独立请求，但不会让同一地址上必须依次生效的更新同时完成。如果原子热点已经饱和，继续增加等待线程主要会增加排队长度。优化可以从两方面着手：缩短服务时间，或者减少需要到达同一个公共地址的更新数量。

3.2 从显存到二级缓存和共享内存

若原子读—改—写必须经过动态随机存取存储器（DRAM），一次更新需要取得旧值并提交新值，长距离数据传输和存储访问会使完整服务时间较大。在以完整读、写往返计时的模型中，这包含读延迟、写延迟以及内部路由和传输开销。对同地址冲突操作，后续更新还必须等待当前更新的完整结果。

二级缓存（L2 cache）可以缩短这一路径。目标位置驻留在 L2 时，支持相应原子处理的硬件可以在更近的位置完成更新，避免完整的 DRAM 往返。不同线程块仍然访问同一公共全局对象；如果它们都更新同一桶，冲突仍存在，只是每次服务可能更快。

共享内存（shared memory）属于线程块，存储距离更近，原子更新通常具有更短的服务路径。它不能让所有线程块直接访问同一个共享数组。要用共享内存统计整份输入，需要让各块各自维护局部直方图，随后把局部结果合并到全局输出。因此，从 L2 的硬件加速进一步走向共享内存，需要算法配合。

这里的层次比较说明机制，不为某一设备指定固定延迟。实际性能还依赖原子数据类型、目标分布、缓存状态和存储系统实现。无论在哪个层次，同一个计数器仍必须表现为一串正确的原子更新。

表 4 三种原子更新位置及其作用

更新位置	延迟来源与改进	仍需处理的限制
DRAM 路径	读、写往返与内部传输构成较长服务过程	同地址排队，完整服务时间较长
L2 路径	目标驻留缓存时可避免 DRAM 往返	全部线程块仍可能争用同一公共桶
共享内存	块内就近更新局部桶	块内同桶更新仍串行，需要跨块合并

3.3 性能优化的两个方向

缩短服务时间时，更新次数可以完全不变。例如，把一个块的 n_r 次更新转移到共享内存后，这些输入仍各贡献一次加一，但每次操作经过较短的局部路径。减少公共更新次数时，则先把多次相同目标的贡献合成局部计数，再向公共桶提交一个较大的增量。

设某线程块处理的输入中有 400 个值等于 7。直接算法会对公共桶 7 做 400 次原子加一；局部统计可以先在该块中得到计数 400，最后只对公共桶 7 做一次原子加 400。每个输入的贡献仍然保留，而公共地址只接收一次更新。共享内存私有化同时利用了更近的局部存储和较少的公共更新。

评价这种变换时，要计入新增的局部数组清零、阶段同步和结果合并。它是否划算取决于每个块能把多少输入更新聚合成多少个非零桶。下一节以完整算法和操作数量明确这些条件。

4 线程块私有化与共享内存

4.1 局部统计与全局结果的等价性

私有化（privatization）为不同参与者维护独立的输出副本，先在副本中积累局部结果，再把这些结果合并到公共输出。在直方图中，适合让每个线程块维护一份共享内存直方图。这里的“私有”是相对于其他线程块而言；同一块内的线程仍共同访问该副本。

令第 r 个线程块处理的输入索引集合为 $\mathcal{I}_r$，其中 $r = 0, \dots, G-1$。这些集合互不相交，且并集包含全部输入。该块的局部直方图记为 $H_j^{(r)}$，则

$$H_j^{(r)} = \sum_{i \in \mathcal{I}_r} \mathbf{1}\{b(x_i) = j\}, \qquad H_j = \sum_{r=0}^{G-1} H_j^{(r)}. \tag{4}$$

第二个等式来自对输入求和的重新分组：每个输入只属于一个集合，所以它对相应桶的那一次贡献只出现一次。局部累加改变了加法的分组方式，不同块完成和合并的顺序也可能不同。

整数计数在不溢出的条件下满足结合律和交换律，因此这两种变化保持结果不变。

更一般地，如果要任意重新分组并调整局部结果的提交顺序，所用合并运算需要满足相应的结合与交换性质。若某个任务要求保留更新的先后顺序，就需要另外保存和执行该顺序。对当前的非负整数计数，逐块合并可以直接使用原子加法。

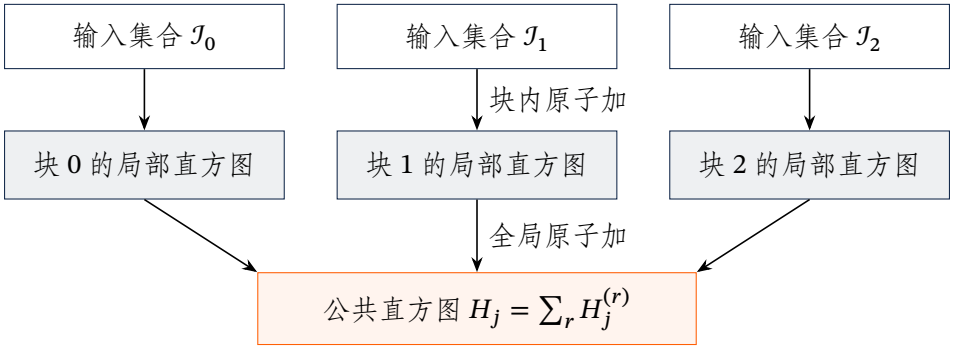


图 1 三个线程块先独立形成局部计数，再逐桶合并到公共结果。

在网格跨步划分下，$\mathcal{I}_r$ 通常由多个起点相隔 GT 的连续小段组成，不必是输入中的一个连续大区间。式(4)只要求集合构成无重复、无遗漏的划分；存储地址上是否连续不影响求和恒等式。

4.2 初始化、局部统计与全局合并

每个线程块执行三个阶段。首先将自己的共享数组清零；然后处理分配到的输入，对该数组中的目标桶做原子加；最后将每个非零局部桶的计数原子加入全局输出。代码4给出完整实现，输入仍为 256 种无符号字节值。

代码 4 共享内存私有化的完整直方图核函数

```
__global__ void histogram_private(
    const unsigned char* input, std::size_t n,
    unsigned int* hist)
{
    __shared__ unsigned int shared_hist[256];

    for (unsigned int b = threadIdx.x; b < 256;
         b += blockDim.x) {
        shared_hist[b] = 0u;
    }
    __syncthreads();

    std::size_t i = std::size_t(blockIdx.x) * blockDim.x
                  + threadIdx.x;
    std::size_t stride = std::size_t(gridDim.x) * blockDim.x;
    for (; i < n; i += stride) {
        atomicAdd(&shared_hist[input[i]], 1u);
    }
    __syncthreads();

    for (unsigned int b = threadIdx.x; b < 256;
         b += blockDim.x) {
        unsigned int count = shared_hist[b];
        if (count != 0u) {
            atomicAdd(&hist[b], count);
        }
    }
}
```

清零阶段按块内线程数跨步分配桶。任一桶 b 都可唯一写成 $b = qT + t$，其中 $0 \le t < T$，因此恰由线程 t 清零。若每块只有 128 个线程，线程 t 依次清零桶 t 和 $t+128$；若每块有 512 个线程，只有前 256 个线程需要写零。这个循环支持任意合法的一维线程块大小。

第一处 `__syncthreads()` 等待块内线程完成全部清零，并使随后阶段能看到这些写入。没有这处同步，快线程可能已经向某个桶加一，负责该桶初始化的慢线程随后再写入零，从而覆盖已有贡献。每个桶只由一个线程清零，也仍然需要阶段间同步，因为清零者与后续更新者可能不同。

局部统计阶段使用共享内存原子加。不同线程块的副本已经分开，因此块 0 对自己桶 7 的更新不会与块 1 的局部桶 7 争用同一地址；但是，同一块内多个线程仍可能同时更新本块的桶 7。共享内存的可见性和地址独立性并不替代块内冲突更新所需的原子性。

第二处 `__syncthreads()` 保证本块所有局部更新完成，随后才读取桶值并提交公共计数。若提前合并，一个线程读到的可能只是局部统计的中间值，后续增量就不会被提交到全局。该屏障位于整个输入循环之后，循环次数不同的线程最终都会到达同一个阶段边界。

4.3 合并阶段的并发与同步边界

第二次同步后，块内不再修改局部直方图。各桶由一个线程读取并负责提交，因此读取局部计数不需要原子操作。不同块却可能同时向同一个全局桶提交结果，所以全局合并仍使用 `atomicAdd`。把局部原子加换成普通加法会在块内丢失更新，把全局原子加换成普通加法则会在块之间丢失更新。

零计数没有贡献，代码通过 `count != 0u` 跳过这类全局更新。提交之后不再复用局部数组，也没有下一阶段需要块内共同读取提交结果，因此无需在核函数末尾再加一次块屏障。不同线程块可以各自完成、各自合并；所有块的更新都结束时，公共直方图才是完整结果。

对这一核函数，不能因为某个线程的初始输入索引超过 $N-1$ 就在开头提前返回。即使它没有输入需要统计，也可能承担某些桶的清零或归并工作，并且需要参与块内的阶段同步。正确写法是让边界条件只控制输入循环的工作量，全部线程继续经过两处屏障。

主机端仍需要在核函数开始前清零全局输出。代码3的清零和完成处理原则可以直接沿用，只需把启动目标改为 `histogram_private`。不能由某个普通线程块在同一个核函数开始处负责清零公共数组、其他块立即累加，因为普通的块内屏障无法保证其他块等到全局清零完成。

算法的正确性由三部分组成：网格跨步划分使每个输入只处理一次；两次屏障和共享原子加使各块准确得到式(4)中的局部计数；全局原子加将全部局部计数完整累加。初始化、阶段边界和原子更新分别保证不同环节。

4.4 原子操作数量与资源开销

令 U_r 表示线程块 r 的非零桶数。直接算法为每个输入执行一次全局原子加，因此全局原子更新总数为 N。私有化算法则执行 N 次共享内存原子加，并在合并阶段执行

$$A_{\text{global}} = \sum_{r=0}^{G-1} U_r \le \min\{N, GB\} \tag{5}$$

次全局原子加。每块最多有 B 个非零桶，所以有上界 GB；每个非零桶至少需要一个输入贡献，而不同块的输入互不重叠，所以同时有上界 N。代码若不跳过零桶，合并阶段的全局原子加次数就固定为 GB。

当各块几乎覆盖全部桶时，可以取 $U_r \approx B$，从而全局原子操作次数由 N 降到约 GB。令每块平均处理的输入数为 $n_{\text{block}} = N/G$，则次数缩减倍数约为 $N/(GB) = n_{\text{block}}/B$。每块处理的数据远多于桶数时，可以把很多局部增量聚合成少量公共更新。

表 5 基本算法与块级私有化的工作量比较

操作或资源	基本全局算法	共享内存私有化
输入元素读取	N	N
共享内存原子加	0	N
全局原子加	N	$\sum_r U_r \le \min\{N, GB\}$
私有桶清零	0	GB
每块屏障次数	0	2
每块共享计数器容量	0	B 个计数器

若各块只处理很少的输入，局部非零桶数可能接近局部输入数，此时 $\sum_r U_r$ 接近 N，而清零、共享原子加和屏障已经引入额外开销。若输入集中于一个桶，只要每个块都有有效输入，就有 $U_r = 1$，全局原子更新数量从 N 降为 G；同时各块仍各自承担局部同桶更新的串行链。

表5统计的是操作数量，不能把不同位置的原子加按相同单价换算成运行时间。是否获得加速还取决于共享内存服务能力、L2 中公共计数器的处理能力、输入分布和设备并行度。次数分析提供的是优化原因与规模关系。

使用 32 位无符号计数器时，每个局部直方图占用 $4B$ 字节；$B = 256$ 时为 1024 字节，即 1 KiB。若某流式多处理器（streaming multiprocessor，SM）可供驻留线程块使用的共享内存为 S_{SM}，仅考虑这一资源时，驻留块数至多为 $\lfloor S_{\mathrm{SM}}/(4B) \rfloor$。实际还受每块和每个 SM 的线程数、寄存器及块数限制，因此局部数组增大可能降低驻留并行度。

4.5 桶数超过共享内存容量时

共享内存私有化要求局部数组能容纳在设备允许的每块共享内存中，并保留足够的并行执行资源。桶数很大时，可以只为一部分桶建立局部数组，按桶区间分批统计。每一批都扫描输入，仅对当前桶区间内的元素更新。这样减少了单批的局部容量，但会重复读取输入；若共有 K 批且每批都完整扫描，输入读取量变为 KN。

另一个选择是把各块的局部直方图放在全局内存，使用不同的行保存不同块的结果。这样仍把原来所有块对同一地址的竞争分散到不同地址，但局部更新不再享受共享内存的低延迟。第一阶段结束后，第二个核函数按桶求各行之和；若每个最终桶由唯一线程或唯一协作组负责输出，最终写入可以不使用全局原子加。其代价是约 GB 个额外计数器、局部结果的全局读写和第二次核函数启动。

这两种方法分别用更多输入扫描或更多全局存储换取较小的共享内存需求。选择时应结合桶数、输入长度和冲突程度；它们保持的数学关系仍然是式(4)。

5 线程粗化与性能权衡

5.1 增加每个线程处理的输入数

线程粗化（thread coarsening）让一个线程承担多个原先可以分配给不同线程的输入元素。在已经使用网格跨步循环的直方图中，可以保持每块线程数 T 不变，减小网格块数 G，让每个线程自然执行更多轮循环。每个块仍只建立一份私有直方图，结束时也只合并这一份结果。

设希望每个线程大致处理 c 个元素，一种网格选择为

$$G_c = \left\lceil \frac{N}{cT} \right\rceil, \qquad P_c = G_c T. \tag{6}$$

该公式用于选择工作划分；末尾不足一轮由边界条件处理。只要 $N > 0$，线程 g 继续按 $i = g + kP_c$ 遍历即可。每一轮中相邻线程仍读取相邻地址，因此这种粗化保留了输入访问的合并模式。

在固定输入长度下，增加 c 使私有直方图数量由约 N/T 减为约 $N/(cT)$。清零量和不跳过零桶时的归并量从约 NB/T 降为 $NB/(cT)$。若跳过零桶，确切归并量仍是 $\sum_r U_r$，但它的 $G_c B$ 上界随块数减少。

这种收益来自实际减少线程块数量。仅把循环展开、让代码一次写出多次更新，而启动的块数保持不变，并不会减少局部副本数量及每块合并开销。另一方面，循环变长也不会减少总输入读取和总共享原子加：它们仍各为 N 次。

5.2 粗化的数量例子与并行度代价

设 $N = 2^{20} = 1\,048\,576$ 个输入，$B = 256$ 个桶，每块 $T = 256$ 个线程。表6比较 $c = 1$ 与 $c = 8$ 的网格设置。两种设置都沿用代码4，没有改变桶数或局部数组类型。

表 6 固定输入规模下减少块数的效果

数量	$c = 1$	$c = 8$
线程块数 G_c	4096	512
每线程处理的输入数	1	8
平均每块输入数	256	2048
私有桶清零次数 $G_c B$	1 048 576	131 072
全局合并次数的 $G_c B$ 上界	1 048 576	131 072
共享内存原子加次数	1 048 576	1 048 576
每块共享直方图容量	1 KiB	1 KiB

清零工作量和全局合并上界下降为原来的八分之一，每个块能用同一份共享数组聚合更多贡献。对于全部输入集中于一个桶的极端情况，实际全局归并次数分别为 4096 和 512。对于分散输入，实际次数由各块观察到的非零桶数决定，不能把表中的上界直接当作测得次数。

增加粗化程度也会让单个块持续更久。如果所有输入集中于一个桶，每块内部对该桶的串行链随局部输入数增加；若整个网格只剩很少几个块，一些 SM 可能没有足够工作，设备无法利用全部并行资源。因而不能仅凭全局归并次数最少，就断定最快的设置应当只使用一个线程块。

通过延长同一个跨步循环实现粗化时，不必然需要更多寄存器，因为循环中可以反复使用同一组索引和临时变量。若具体实现同时保留多组输入、中间结果或展开后的独立状态，才可能增加寄存器压力。粗化的资源代价需要对应到实际代码，而它减少私有副本数量的机制由网格规模直接决定。

5.3 线程级局部聚合

粗化使一个线程依次处理多个输入，也提供了在线程内部合并更新的机会。若为每个线程保存完整直方图，需要每线程 B 个计数器；桶数较大时，寄存器或局部存储开销可能过高。一个更小的状态只保存最近访问的桶编号和该桶连续出现的次数。线程再次读到同一桶时只增加私有计数，读到不同桶时才把上一段计数原子加入块内直方图。

设线程 g 实际读取的桶序列为 $b_{g,0},\dots,b_{g,m_g-1}$。将其中连续相同的桶划为一段，段数为

$$R_g=\begin{cases}0, & m_g=0,\\ 1+\displaystyle\sum_{k=1}^{m_g-1}\mathbf{1}\{b_{g,k}\neq b_{g,k-1}\}, & m_g>0.\end{cases} \tag{7}$$

每一段只提交一次，因此共享内存原子加的次数由 N 变为 $\sum_g R_g\le N$。每个桶的贡献总量不变，块内直方图及其非零桶数也不变，所以最后的全局合并次数仍为 $\sum_r U_r$。

以下伪代码替换共享直方图核函数中的输入统计循环；前面的清零与第一处屏障，以及后面的第二处屏障与全局合并，均保持不变。`valid` 表示是否存在尚未提交的一段，不能用某个合法桶编号表示空状态。

代码 5 线程内合并连续同桶更新的伪代码

```
valid = false
count = 0
for each valid input index assigned to this thread:
    bin = input[index]
    if valid and bin == last_bin:
        count = count + 1
    else:
        if valid:
            atomicAdd(shared_hist[last_bin], count)
        last_bin = bin
        count = 1
        valid = true
if valid:
    atomicAdd(shared_hist[last_bin], count)
// All threads then reach the block barrier.
```

线程读到的序列若为 $(7,7,2,2,2,7)$，就依次提交 $(7,2)$、$(2,3)$ 和 $(7,1)$ 三个“桶、增量”对，将六次共享原子加减少为三次。最后一对在循环结束后提交；省略这一步会丢失末段。空线程不提交任何计数，但仍参加块内屏障；只处理一个元素的线程则通过末段提交完成唯一一次更新。私有段计数和共享计数器同样必须能表示实际累计数量。

这里的连续性取决于线程的访问顺序。网格跨步循环使同一线程依次读取 $g, g+P, g+2P, \ldots$，原数组中相邻像素的相似性不保证这些位置也相似。例如，输入 $(0,0,1,1,0,0,1,1)$ 由 $P=2$ 个线程跨步读取时，每个线程都看到 $(0,1,0,1)$，八次原子更新一次也不能省略；若改成 $(0,1,0,1,0,1,0,1)$，两个线程分别看到四个 0 和四个 1，只需两次提交。输入的桶频数在这两个例子中相同，聚合收益却不同。

当线程所见序列包含较长的同桶段时，少量私有状态能够明显减少热点桶上的共享原子操作。若相邻访问很少命中同桶，R_g 接近 m_g，原子操作数量几乎不变，而比较、分支与状态维护仍有成本。因此，线程级聚合应结合实际访问顺序和输入分布评估，不能仅凭直方图中存在高频桶就断定会加速。

5.4 访存、分支和复用的整体判断

输入数组通常只被读取一次，每个元素只用于决定自己的桶，因此算法没有需要专门缓存的跨线程输入复用。私有直方图则不同：同一个块内的许多输入可能反复更新同一组桶，输出状态具有复用。将这组状态放进共享内存，正是把访问频繁的数据保留在块内。

输入跨步循环的边界条件主要影响最后一轮不满的线程束，或者没有输入任务的线程。合并阶段的非零判断也可能使一部分线程提交更新、另一部分跳过；这种分支换来了对零贡献原子操作的省略。相比之下，很多线程同时命中同一桶造成的是原子地址竞争：即使所有线程执行完全相同的指令，也可能因为目标地址相同而排队。

表 7 私有化直方图的主要性能因素

观察对象	形成的开销	对应调整
同轮输入地址	地址分散时需要更多内存事务	使相邻线程读取相邻输入
公共桶竞争	多块贡献排成同地址更新序列	先在块内聚合，再提交局部计数
块内热点桶	共享内存同址原子更新仍排队	结合局部输入量与数据分布评估
副本数量	每块都需清零和逐桶合并	适度增加每块工作量、减少块数
驻留与并行度	资源需求大或网格过小会限制并行	同时考虑桶容量、线程数和块数

直接全局原子、共享内存私有化和线程粗化分别改变不同环节。基本算法提供简单的正确实现；私有化改变竞争发生的位置及公共更新次数；粗化进一步摊薄每份私有副本的固定开销。比较这些方案时，应固定输入长度、桶映射和数据分布，再结合操作数量与实际运行时间判断收益。

6 例题与完整推导

6.1 从输入划分到最终直方图

设输入长度为 $N = 16$，数据依次为

$$x = (0, 2, 1, 2,\ 3, 2, 0, 1,\ 2, 3, 3, 1,\ 0, 2, 3, 2).$$

使用两个线程块，每块四个线程，即 $G = 2$、$T = 4$、$P = 8$，运行代码4。桶数仍为 256，但只有桶 0 到 3 可能非零。要求写出各线程负责的输入、每块的局部结果、全局结果，并计算原子操作次数。

线程块 r 中线程 t 的全局编号为 $g = 4r + t$，处理索引 g 与 $g + 8$。表8逐一列出两个轮次，其中 $i \mapsto x_i$ 表示“输入索引映射到该位置的值”。

表 8 两块、每块四线程的完整输入分配

块编号 r	块内线程 t	第一轮 $i \mapsto x_i$	第二轮 $i \mapsto x_i$
0	0	$0 \mapsto 0$	$8 \mapsto 2$
0	1	$1 \mapsto 2$	$9 \mapsto 3$
0	2	$2 \mapsto 1$	$10 \mapsto 3$
0	3	$3 \mapsto 2$	$11 \mapsto 1$
1	0	$4 \mapsto 3$	$12 \mapsto 0$
1	1	$5 \mapsto 2$	$13 \mapsto 2$
1	2	$6 \mapsto 0$	$14 \mapsto 3$
1	3	$7 \mapsto 1$	$15 \mapsto 2$

块 0 处理的八个值为 $0, 2, 1, 2, 2, 3, 3, 1$，所以桶 0 到 3 的局部计数为 $H^{(0)} = (1, 2, 3, 2)$。块 1 处理的值为 $3, 2, 0, 1, 0, 2, 3, 2$，局部计数为 $H^{(1)} = (2, 1, 3, 2)$。每个局部向量的元素之和都是 8，符合各块分配的输入数。两块分别完成第二次同步后，逐桶合并得到

$$H = H^{(0)} + H^{(1)} = (1, 2, 3, 2) + (2, 1, 3, 2) = (3, 3, 6, 4). \tag{8}$$

这里为了简洁只列出前四个桶，其余 252 个桶为零。总和 $3 + 3 + 6 + 4 = 16$，且逐桶结果与直接计数相同。

直接算法需要 16 次全局原子加。私有化算法需要 16 次共享内存原子加，每块有 4 个非零桶，所以全局合并只做 $4 + 4 = 8$ 次原子加。由于代码仍为每块清零 256 个桶，局部清零总数却是 $2 \times 256 = 512$ 次；这个小例子说明，全局原子次数减少并不足以保证整体加速，私有

化还需要足够工作量来摊薄固定开销。

如果输入只保留前 14 个元素，索引 14 和 15 不再处理。块 1 的线程 2、3 在第二轮没有有效输入，但仍参与局部初始化和两次同步。最终前四桶为 $(3,3,5,3)$，总计数为 14。这一边界变化只减少两次输入贡献，不改变线程块各阶段的参与规则。

6.2 由浮点吞吐率反推 L2 原子操作比例

设一次原子操作在 L2 完成时耗时 5 ns，在 DRAM 完成时耗时 120 ns；一次浮点运算耗时 1 ns。每个线程执行 5 次原子操作和 100 次浮点运算，因此每次原子操作对应 20 次浮点运算。给定有效浮点吞吐率为 0.2424 GFLOPS，其中 1 GFLOPS 表示每秒 10^9 次浮点运算。求在 L2 完成的原子操作所占比例。

采用等效单条串行执行流的简化计时模型：一个线程所代表的工作量中，L2 原子操作、DRAM 原子操作和浮点运算的用时直接相加；给定吞吐率按这一等效工作流换算。令 $p \in [0,1]$ 表示原子操作在 L2 完成的平均比例，则一次原子操作的平均耗时为 $5p+120(1-p)$ 纳秒。一份线程工作量的平均耗时为

$$t(p) = 5\big[5p + 120(1-p)\big] + 100 = 700 - 575p \quad \text{ns}. \tag{9}$$

式中最外侧的 5 是每线程原子操作次数，括号里的 5 是一次 L2 原子操作的纳秒耗时；最后的 100 是全部浮点运算的纳秒耗时。

将给定吞吐率换算为每秒完成的线程工作量：每份工作量含 100 次浮点运算，所以速率为 $(0.2424 \times 10^9)/100 = 2.424 \times 10^6$ 份工作量每秒。取倒数得到平均用时

$$t_{\text{obs}} = \frac{100}{0.2424 \times 10^9}\ \text{s} \approx 412.5413\ \text{ns}. \tag{10}$$

在这里，一次浮点运算每纳秒等于 1 GFLOPS，也可以直接计算 100/0.2424，并将所得时间单位写为纳秒。

令式(9)等于式(10)，可得

$$p = \frac{700 - 412.5413}{575} \approx 0.49993 \approx 50\%. \tag{11}$$

给定吞吐率只有四位小数，因此答案写作约 50%。若取 $p = 1/2$ 反向核验，则平均原子操作耗时为 $(5+120)/2 = 62.5$ ns；每份工作量耗时 $5 \times 62.5 + 100 = 412.5$ ns，对应 $100/412.5 = 0.242424\ldots$ GFLOPS，四舍五入即为 0.2424。

还可以检查两个端点：若全部原子操作在 L2 完成，平均耗时为 125 ns，吞吐率为 0.8 GFLOPS；若全部在 DRAM 完成，平均耗时为 700 ns，吞吐率约为 0.142857 GFLOPS。给定值位于二者之间，与求得的 $p \in [0,1]$ 一致。50% 是许多线程或多次执行中原子操作的平均比例，不要求每个含五次原子操作的线程恰好有同样的 L2 次数。

这个反推依赖已明确的串行等效计时口径。只有“单线程内部三类操作不重叠”，还不足以把真实多线程设备的总吞吐直接换算成单线程延迟；线程之间可能重叠执行，也可能共享原子服务资源并排队。若用于真实内核测量，需要额外确定并行执行与吞吐统计的关系，才能由总吞吐唯一反推存储层级比例。

6.3 实现检查与关键关系

对直方图实现，可以围绕完整的贡献路径检查正确性：每个输入是否恰好被处理一次，桶下标是否有效，每次共享更新是否被保留，以及结果读取是否发生在相关更新完成之后。表9列出四种具体错误及其后果。

表 9 典型实现错误及其修正依据

错误写法或条件	产生的问题	修正依据
每块 128 线程，仅用小于 256 的线程编号清零同编号桶	只有桶 0 到 127 被清零，后半数组初值未知	按块内线程数跨步遍历全部桶
共享数组上的累加改成普通加一	同块多个线程命中同一桶时丢失更新	局部冲突仍需共享原子加
第二次同步前读取局部计数并合并	局部统计尚未结束，后续贡献可能遗漏	所有局部更新结束后再提交结果
多个块用普通加法合并到同一公共桶	不同块的局部总数可能互相覆盖	使用全局原子加，或另设独占输出的归并阶段

数学结果始终由 $H_j = \sum_i \mathbf{1}\{b(x_i) = j\}$ 定义。网格跨步划分通过 $i = g + kGT$ 保证输入覆盖；块级私有化通过 $H_j = \sum_r H_j^{(r)}$ 保持结果；跳过零桶时，全局更新次数为 $\sum_r U_r \le \min\{N, GB\}$。这些关系分别连接输入分配、统计正确性和性能开销。

合并访存改善同一轮输入读取，原子操作保护冲突的读—改—写，私有化把许多公共更新聚合为局部总数，线程粗化则用更长的局部工作摊薄初始化与合并成本。将这几个机制放回各自负责的环节，才能解释为什么一个实现既正确、又在特定数据规模和分布下更高效。